C# Programming for Indian Students — Chapter 3: C# Fundamentals

Oct 31, 2026 · @Vikash

3.1 What you will learn

So far your programs only printed fixed text. Real programs also **remember** things: a student's name, a roll number, a price. This chapter shows how C# stores such information in memory. By the end you will be able to:

- declare a variable, give it a value and change that value later;
- choose the right data type for a name, a whole number, a price, a grade or a yes/no answer;
- write literal values correctly, including the suffixes `L`, `f` and `m`;
- follow the naming rules for variables and use the standard C# style;
- create a constant with `const` and let the compiler work out a type with `var`;
- print variables neatly using string interpolation;
- read and fix the most common compiler errors about variables.

Program input from the keyboard and calculations come in Chapter 4. Here every value is typed into the program itself.

Remember: all programs in this chapter use top-level statements, so each one is just a list of lines you can place in `Program.cs` of a new console project.

Vikash Kumar

3.2 Variables

A variable is a named place in memory that holds a value, and the value can change while the program runs.

Analogy. Think of a row of lockers in your college. Each locker has a **label** (its name), only fits one **kind of thing** (its type) and holds one **item** at a time (its value). You can open a locker and replace the item with another of the same kind, but you cannot put a cricket bat in a locker made for a notebook.

```
+-----------+   +-----------+
|  name     |   |  age      |
|  string   |   |  int      |
|  "Aarav"  |   |  18       |
+-----------+   +-----------+
 locker 1         locker 2
```

Syntax. A variable is created with a type, a name and (usually) a starting value.

```
type name = value;
```

Term	Meaning	Example
Declaration	Creating the variable: type and name	`int age;`
Initialisation	Giving it its first value	`int age = 18;`
Assignment	Putting a new value in later	`age = 19;`

The single `=` sign is the **assignment operator**. It does not mean "equal to" as in mathematics. It means: *take the value on the right and store it in the variable on the left.*

Program 3.1: Declare, print and change a variable

```
int age = 18;
string name = "Aarav";
Console.WriteLine(name);
Console.WriteLine(age);
age = 19;
Console.WriteLine(age);
```

Output

```
Aarav
18
19
```

How it works. The first line creates an `int` variable called `age` and stores 18. The second creates a `string` variable called `name`. After printing both, `age = 19;` replaces the old value 18 with 19. The old value is gone, and the last line prints the new one. Notice there are **no quotes** around `name` and `age` inside `WriteLine`: with quotes C# would print the word itself, and without quotes it prints the value stored in the variable.

Remember: a variable name can be reused for new values, but its type is fixed for ever. An `int` variable can never hold text.

Vikash Kumar

3.3 Data types

A **data type** tells C# what kind of value a variable holds and how much memory it needs. Choosing the right type makes programs correct and efficient.

Type	Stores	Size	Example value	Range or note
`byte`	Very small whole number	1 byte	`200`	0 to 255
`short`	Small whole number	2 bytes	`30000`	-32,768 to 32,767
`int`	Whole number (most common)	4 bytes	`1400000000`	-2,147,483,648 to 2,147,483,647
`long`	Very large whole number	8 bytes	`9460730472580800L`	about -9.2 to +9.2 quintillion
`float`	Decimal number, about 7 digits	4 bytes	`36.6f`	Less precise, saves memory
`double`	Decimal number, about 15 digits	8 bytes	`3.14159`	Default for decimals
`decimal`	Exact decimal number, about 28 digits	16 bytes	`499.99m`	Use for money
`char`	One character	2 bytes	`'A'`	Single quotes
`bool`	True or false	1 byte	`true`	Only two values
`string`	Text	Depends on the length	`"Pune"`	Double quotes

Vikash Kumar

Program 3.2: One variable of each type

```
byte smallNumber = 200;
short shortNumber = 30000;
int population = 1400000000;
long distance = 9460730472580800L;
float temperature = 36.6f;
double pi = 3.141592653589793;
decimal price = 499.99m;
char grade = 'A';
bool isPassed = true;
string college = "Government Engineering College";
```

```
Console.WriteLine("byte    : " + smallNumber);
Console.WriteLine("short   : " + shortNumber);
Console.WriteLine("int     : " + population);
Console.WriteLine("long    : " + distance);
Console.WriteLine("float   : " + temperature);
Console.WriteLine("double  : " + pi);
Console.WriteLine("decimal : " + price);
Console.WriteLine("char    : " + grade);
Console.WriteLine("bool    : " + isPassed);
Console.WriteLine("string  : " + college);
```

Output

```
byte    : 200
short   : 30000
int     : 1400000000
long    : 9460730472580800
float   : 36.6
double  : 3.141592653589793
decimal : 499.99
char    : A
bool    : True
string  : Government Engineering College
```

The `+` between a piece of text and a variable joins them into one line of output. This is called **concatenation**. Note that C# prints `bool` values as `True` and `False` with a capital first letter.

Finding the limits of a type. Every numeric type has `MinValue` and `MaxValue`, and `sizeof` gives its size in bytes.

Program 3.3: Ranges and sizes

```
Console.WriteLine("int  : " + int.MinValue + " to " + int.MaxValue);
Console.WriteLine("long : " + long.MinValue + " to " + long.MaxValue);
Console.WriteLine("byte : " + byte.MinValue + " to " + byte.MaxValue);
Console.WriteLine("Size of int    : " + sizeof(int) + " bytes");
Console.WriteLine("Size of long   : " + sizeof(long) + " bytes");
Console.WriteLine("Size of double : " + sizeof(double) + " bytes");
Console.WriteLine("Size of char   : " + sizeof(char) + " bytes");
Console.WriteLine("Size of bool   : " + sizeof(bool) + " byte");
```

Output

Vikash Kumar

```
int  : -2147483648 to 2147483647
long : -9223372036854775808 to 9223372036854775807
byte : 0 to 255
Size of int    : 4 bytes
Size of long   : 8 bytes
Size of double : 8 bytes
Size of char   : 2 bytes
Size of bool   : 1 byte
```

Which type should I use?

Information	Best type	Why
Age, roll number, quantity	`int`	Whole numbers, no fractions
Population of a country	`int` or `long`	`int` holds up to about 2.1 billion
Price, fee, salary	`decimal`	Exact, so paise are never lost
Percentage, CGPA, height	`double`	Fractions where tiny rounding is fine
Grade (A, B, C)	`char`	One character
Passed or failed, hosteler or not	`bool`	Only two possible answers
Name, city, college	`string`	Text
Mobile number, PIN code, Aadhaar number	`string`	You never add them, and a leading zero must not be lost

Remember: `char` uses single quotes (`'A'`) and `string` uses double quotes (`"A"`). They are different types even for one letter.

3.4 Literals, naming rules and var

Literals and suffixes. A **literal** is a fixed value written directly in the code, such as `18` , `3.14` , `'A'` or `"Pune"` . C# decides the type of a number from how you write it: a whole number is an `int` , and a number with a decimal point is a `double` . To choose another type you add a **suffix** letter.

Suffix	Type	Example
(none)	`int` for whole numbers	`25`
(none)	`double` for decimals	`3.14`

Vikash Kumar

Suffix	Type	Example
`L`	`long`	`9460730472580800L`
`f` or `F`	`float`	`36.6f`
`m` or `M`	`decimal`	`499.99m`

Rules for naming a variable. A name is called an **identifier**.

1. It may contain letters, digits and the underscore `_`.
2. It must **not start with a digit**.
3. It cannot contain spaces or symbols such as `-`, `@` or `#`.
4. It cannot be a **keyword**, a word that C# reserves, such as `int`, `class`, `string`, `static`, `void` or `if`.
5. It is **case-sensitive**: `age`, `Age` and `AGE` are three different variables.

Name	Valid?	Reason
`studentName`	Valid	Letters only
`marks1`	Valid	Digit allowed after the first character
`_total`	Valid	May start with an underscore
`2ndYear`	Invalid	Starts with a digit
`total marks`	Invalid	Contains a space
`class`	Invalid	Keyword

Naming style. Microsoft recommends **camelCase** for variables (first word small, next words start with a capital: `studentName`, `totalMarks`) and **PascalCase** for constants and class names (`Pi`, `DaysInWeek`). Choose names that explain the purpose: `age` is better than `a`, and `totalMarks` is better than `tm`.

Letting C# find the type with `var`. When you give a value at the time of declaration, you can write `var` and the compiler works out the type from the value. The variable still has one fixed type after that.

```
var city = "Pune";    // string
var count = 5;        // int
var price = 99.5;     // double
```

We checked these three by asking C# for each type name. The output is `String Int32 Double`, which are the .NET names for `string`, `int` and `double`.

Vikash Kumar

Remember: `var` is not a new type and it is not "any type". It needs a value on the same line so that the compiler can decide. In this book, beginners' programs mostly write the type explicitly, because it is easier to read and to learn from.

3.5 Constants and printing variables

Constants. A **constant** is a value that must never change while the program runs, for example the number of days in a week. You create one by adding the word `const` and you must give its value at once. The compiler then refuses any attempt to change it, which protects you from mistakes.

```
const double Pi = 3.14159;
const int DaysInWeek = 7;
Console.WriteLine("Pi = " + Pi + ", days = " + DaysInWeek);
```

Output

```
Pi = 3.14159, days = 7
```

Printing with string interpolation. Joining pieces with `+` quickly becomes untidy. A better way is **string interpolation**: put a `$` before the opening quote, and write variable names inside curly braces `{ }`. C# replaces each `{name}` with the value.

Program 3.4: Printing variables

```
string name = "Aarav";
int roll = 21;
double cgpa = 8.456;
char grade = 'A';
bool isHosteler = false;
decimal fee = 45000.50m;

Console.WriteLine("Name: " + name);
Console.WriteLine($"Roll No: {roll}");
Console.WriteLine($"CGPA: {cgpa:F2}");
Console.WriteLine($"Grade: {grade}");
Console.WriteLine($"Hosteler: {isHosteler}");
Console.WriteLine($"Fee: Rs. {fee:F2}");
Console.WriteLine($"{name} (Roll No. {roll}) scored grade {grade}.");
```

Output

Vikash Kumar

```
Name: Aarav
Roll No: 21
CGPA: 8.46
Grade: A
Hosteler: False
Fee: Rs. 45000.50
Aarav (Roll No. 21) scored grade A.
```

Format specifier. `{cgpa:F2}` means "show this number with exactly **2** digits after the decimal point". The `F` stands for *fixed-point*. The value 8.456 was rounded to 8.46 for display, but the variable still holds 8.456. On a computer set to some other region, the decimal point may appear as a comma, because C# follows the computer's regional settings.

Style	Example	When to use
Concatenation	`"Name: " + name`	Very short lines
Interpolation	`$"Name: {name}"`	Most printing, easier to read and format

Remember: constants use PascalCase (`DaysInWeek`), and interpolation needs the `$` before the quote. Without the `$` , C# prints the curly braces literally.

3.6 Dry run and common errors

Dry run: variables change one line at a time. Follow this program and write the value of each variable after every line.

Program 3.5

```
int a = 10;
int b = a;
a = 20;
Console.WriteLine("a = " + a);
Console.WriteLine("b = " + b);
```

Step	Statement	Value of `a`	Value of `b`	Screen
1	`int a = 10;`	10	(not created yet)	
2	`int b = a;`	10	10 (a copy of a)	
3	`a = 20;`	20	10	

Vikash Kumar

Step	Statement	Value of `a`	Value of `b`	Screen
4	`Console.WriteLine("a = " + a);`	20	10	`a = 20`
5	`Console.WriteLine("b = " + b);`	20	10	`b = 10`

Output

```
a = 20
b = 10
```

Step 2 copies the **value** of `a` into `b`. After that the two variables are independent, so changing `a` in step 3 does not change `b`.

Common errors. Each message below is the real message the compiler printed for that code.

Mistake	Wrong code	Compiler message	Fix
Name starts with a digit	`int 2ndYear = 2;`	`error CS1001: Identifier expected`	Start with a letter: `int secondYear = 2;`
Decimal stored in `float` without `f`	`float f = 3.14;`	`error CS0664: Literal of type double cannot be implicitly converted to type 'float'; use an 'F' suffix to create a literal of this type`	Write `3.14f`, or use `double`
Using a variable before giving it a value	`int x;` then `Console.WriteLine(x);`	`error CS0165: Use of unassigned local variable 'x'`	Write `int x = 0;`

Vikash Kumar

Mistake	Wrong code	Compiler message	Fix
Changing a constant	`const int Days = 7;` then `Days = 8;`	`error CS0131: The left-hand side of an assignment must be a variable, property or indexer`	Do not change a `const`; use a normal variable
Wrong type of value	`int age = "eighteen";`	`error CS0029: Cannot implicitly convert type 'string' to 'int'`	Write `int age = 18;`
Same name declared twice	`int age = 18;` then `int age = 19;`	`error CS0128: A local variable or function named 'age' is already defined in this scope`	Declare once, then assign: `age = 19;`
Value too large for the type	`byte b = 300;`	`error CS0031: Constant value '300' cannot be converted to a 'byte'`	Use `short` or `int`
Two characters in a `char`	`char c = 'AB';`	`error CS1012: Too many characters in character literal`	Use a `string`: `"AB"`
Wrong capital letter	`string Name = "A";` then `Console.WriteLine(name);`	`error CS0103: The name 'name' does not exist in the current context`	Use the same spelling and case everywhere

Remember: read the message and the line number the compiler gives you. In Visual Studio, double-click the error in the **Error List** to jump to the exact place.

Vikash Kumar

3.7 Mini project: Student Profile Card

Problem. Your college office wants a program that prints a neat profile card for a student. The program keeps each detail in a variable of the right type, then prints the card. This uses every idea of the chapter: types, a constant, and interpolation with a format.

Program 3.6: Student Profile Card

```
string name = "Aarav Sharma";
int rollNo = 21;
char section = 'B';
string course = "BCA";
int semester = 2;
double cgpa = 8.456;
bool isHosteler = false;
decimal annualFee = 45000.50m;
const string College = "Government Engineering College";

Console.WriteLine("==================================");
Console.WriteLine("        STUDENT PROFILE CARD");
Console.WriteLine("==================================");
Console.WriteLine($"College    : {College}");
Console.WriteLine($"Name       : {name}");
Console.WriteLine($"Roll No    : {rollNo}");
Console.WriteLine($"Course     : {course}, Semester {semester}, Section
{section}");
Console.WriteLine($"CGPA       : {cgpa:F2}");
Console.WriteLine($"Hosteler   : {isHosteler}");
Console.WriteLine($"Annual fee : Rs. {annualFee:F2}");
Console.WriteLine("==================================");
```

Output

```
==================================
        STUDENT PROFILE CARD
==================================
College    : Government Engineering College
Name       : Aarav Sharma
Roll No    : 21
Course     : BCA, Semester 2, Section B
CGPA       : 8.46
Hosteler   : False
Annual fee : Rs. 45000.50
==================================
```

Why these types?

Vikash Kumar

Detail	Type	Reason
Name, course	`string`	Text
Roll number, semester	`int`	Whole numbers
Section	`char`	One letter
CGPA	`double`	A decimal where tiny rounding is fine
Hosteler	`bool`	Yes or no
Annual fee	`decimal`	Money must be exact
College	`const string`	It never changes during the run

Try it yourself

1. Replace the details with your own and change the college name.
2. Add a `string` variable for your city and a `bool` for whether you have a laptop.
3. Add a `char` variable for your blood group initial (for example `'O'`) and print it.
4. Print the annual fee with the label `Fee (Rs.)` instead.

3.8 Practice questions

Try each question on your own first. The answers follow the list.

Section A: Short answer

1. What is a variable? Explain with an example.
2. What is the difference between declaration, initialisation and assignment?
3. Which data type would you use for: (a) age, (b) price of a book, (c) a grade letter, (d) whether a student is present, (e) a mobile number?
4. State any four rules for naming a variable.
5. What is the difference between `char` and `string`?
6. What is the use of `const`? Write one example.
7. What is the difference between `float`, `double` and `decimal`?

Section B: Predict the output

8. What does this program print?

Vikash Kumar

```
int x = 5;
int y = x;
x = 8;
Console.WriteLine(x + " " + y);
```

9. What does this program print?

```
string city = "Pune";
int pin = 411001;
Console.WriteLine($"{city} - {pin}");
Console.WriteLine("{city} - {pin}");
```

10. What does this program print?

```
double d = 7.0;
decimal m = 5.50m;
Console.WriteLine(d);
Console.WriteLine(m);
```

Section C: Find the error

11. `int marks = 85.5;`
12. `string name = 'Aarav';`

Section D: Write the program

13. Declare variables for a shop item (name, price, quantity) and print them as three labelled lines using interpolation.

Answers

1. A variable is a named place in memory that holds a value that can change. Example: `int age = 18;` creates a variable `age` that holds 18.
2. Declaration creates the variable (`int age;`). Initialisation gives its first value (`int age = 18;`). Assignment stores a new value later (`age = 19;`).
3. (a) `int` , (b) `decimal` , (c) `char` , (d) `bool` , (e) `string` .
4. Use letters, digits and underscore only; do not start with a digit; no spaces or symbols; not a keyword; names are case-sensitive.
5. `char` holds exactly one character in single quotes, such as `'A'` . `string` holds text of any length in double quotes, such as `"Aarav"` .
6. `const` creates a value that cannot be changed after it is set. Example: `const double Pi = 3.14159;` .

Vikash Kumar

7. `float` is 4 bytes with about 7 digits of precision, `double` is 8 bytes with about 15 digits, and `decimal` is 16 bytes with about 28 digits and is exact for decimal fractions, so it is used for money.
8. Output:

```
8 5
```

`y` got a copy of 5, so changing `x` later does not change `y`.

9. Output:

```
Pune - 411001
{city} - {pin}
```

The second line has no `$`, so the braces are printed as they are.

10. Output:

```
7
5.50
```

A `double` prints `7.0` as `7`, while a `decimal` keeps the digits you wrote in `5.50m`.

11. `error CS0266: Cannot implicitly convert type 'double' to 'int'. An explicit conversion exists (are you missing a cast?)`. An `int` cannot hold 85.5. Use `double marks = 85.5;` (casting is covered in Chapter 4).
12. `error CS1012: Too many characters in character literal`. Single quotes are for one character. Use double quotes: `string name = "Aarav";`.
13. One possible answer:

```
string item = "Pen";
decimal price = 12.50m;
int quantity = 4;
Console.WriteLine($"Item     : {item}");
Console.WriteLine($"Price    : Rs. {price}");
Console.WriteLine($"Quantity : {quantity}");
```

```
Item     : Pen
Price    : Rs. 12.50
Quantity : 4
```

3.9 MCQs and viva questions

Multiple-choice questions (answers are listed after the questions)

Vikash Kumar

1. A variable is:
   - (a) a fixed value that never changes
   - (b) a named place in memory that holds a value
   - (c) a type of loop
   - (d) a comment
2. Which data type is used to store a whole number in C#?
   - (a) `string`
   - (b) `bool`
   - (c) `int`
   - (d) `char`
3. Which of these is a `char` literal?
   - (a) `'A'`
   - (b) `"A"`
   - (c) `A`
   - (d) `(A)`
4. Which is a valid variable name?
   - (a) `2marks`
   - (b) `total marks`
   - (c) `class`
   - (d) `_total`
5. Which keyword creates a constant?
   - (a) `static`
   - (b) `const`
   - (c) `fixed`
   - (d) `final`
6. Which suffix makes a decimal literal such as `499.99`?
   - (a) `f`
   - (b) `L`
   - (c) `m`
   - (d) `d`
7. The type of the literal `3.14` is:
   - (a) `double`
   - (b) `float`
   - (c) `decimal`

Vikash Kumar

   - (d) `int`
8. Putting `$` before a string, as in `$"Hello {name}"`, enables:
   - (a) comments
   - (b) loops
   - (c) constants
   - (d) string interpolation
9. How many bytes does an `int` use?
   - (a) 2
   - (b) 4
   - (c) 8
   - (d) 16
10. The best type for storing money is:
   - (a) `float`
   - (b) `double`
   - (c) `decimal`
   - (d) `char`

MCQ answers: 1-b, 2-c, 3-a, 4-d, 5-b, 6-c, 7-a, 8-d, 9-b, 10-c.

Viva questions (short oral questions with model answers)

1. *Why does C# need data types?* They tell the compiler how much memory a value needs and which operations are allowed, so many mistakes are caught before the program runs.
2. *Can the type of a variable change after it is declared?* No. The value can change, but the type is fixed. Even with `var`, the type is decided once from the first value.
3. *Why is `decimal` preferred for money?* It stores decimal fractions exactly, while `float` and `double` store binary approximations and can show tiny errors.
4. *What is the difference between `=` and `==`?* `=` assigns a value to a variable. `==` compares two values (you will use it in Chapter 5).
5. *What happens if you use a variable before giving it a value?* The compiler reports error CS0165, "Use of unassigned local variable".
6. *What is a keyword?* A reserved word with a special meaning in C#, such as `int` or `class`. It cannot be used as a variable name.
7. *What is the difference between `Console.WriteLine("name")` and `Console.WriteLine(name)`?* The first prints the word name. The second prints the value stored in the variable `name`.

Vikash Kumar

3.10 Summary and what comes next

- A **variable** is a named memory location with a fixed type and a value that can change.
- Create one with `type name = value;`. Use `=` to assign a new value later.
- Common types: `int`, `long`, `double`, `decimal` (money), `char`, `bool` and `string`.
- Use suffixes `L`, `f` and `m` for `long`, `float` and `decimal` literals.
- Variable names use letters, digits and underscore, never start with a digit, are case-sensitive and cannot be keywords. Use camelCase.
- `const` makes a value that cannot change. `var` lets the compiler pick the type from the value.
- Use string interpolation (`$"Name: {name}"`) and format specifiers (`{cgpa:F2}`) to print values neatly.

Next chapter: Chapter 4, **Operators and User Input**, lets your programs calculate and listen. You will use arithmetic, relational and logical operators, read values typed by the user with `Console.ReadLine`, convert text to numbers, and build a GST calculator.

Vikash Kumar